Generating Interval Orders

Gurvan Estable
(supervisor: Uli Fahrenberg
Hugo Bazille)

Technical Report n^o202507-techrep-estable, July 2025
revision 1.0

The exhaustive and efficient generation of all non-isomorphic interval orders is a major challenge. This report presents a generation method that solves this problem by avoiding expensive isomorphism tests. Leveraging the sparse decomposition of interval orders, we introduce a canonical signature that uniquely represents each structure, regardless of element labels. This approach allows us to define "prototypes," irreducible orders that serve as building blocks. Our main contribution is a two-step algorithm that first generates these prototypes and then systematically derives all possible interval orders from them. The method proves effective, enabling the generation of all orders up to 14 elements.

La génération exhaustive et efficace de tous les ordres d'intervalles non-isomorphes est un défi majeur. Ce rapport présente une méthode de génération qui résout ce problème en évitant les coûteux tests d'isomorphisme. En nous appuyant sur la décomposition sparse des ordres d'intervalles, nous introduisons une signature canonique qui représente chaque structure de manière unique, indépendamment des étiquettes de ses éléments. Cette approche nous permet de définir des "prototypes", des ordres irréductibles qui servent de briques de base. Notre contribution principale est un algorithme en deux étapes qui génère d'abord ces prototypes, puis en dérive systématiquement tous les ordres d'intervalles possibles. Cette méthode s'avère efficace, permettant la génération de tous les ordres jusqu'à 14 éléments.

Keywords
poset, iposet, interval order, generation

Laboratoire de Recherche de l'EPITA
14-16, rue Voltaire – FR-94276 Le Kremlin-Bicêtre CEDEX – France
Tél. +33 1 53 14 59 22 – Fax. +33 1 53 14 59 13
gurvan.estable@epita.fr – http://www.lre.epita.fr/

Copying this document

Contents

Chapter 1

Introduction

Partially Ordered Sets (POSETs) are an important mathematical tool for expressing sets that possess some partial order, such as sets describing the divisibility between numbers, for example. In this research, we will particularly focus on a subset of POSETs called interval orders. Interval orders are special POSETs used to describe orders that can be represented by intervals, and they have many concrete applications, typically in concurrency theory or scheduling.

The foundational structure of interval orders has been extensively studied, notably characterized by Fishburn as (2+2)-free posets Fishburn (1985). Building upon this, subsequent research has further explored the algebraic and compositional properties of these structures. Notably, the work of Fahrenberg and his collaborators has advanced the field by introducing the concept of iposets (posets with interfaces), which allows for a compositional understanding of posets Fahrenberg et al. (2020). This framework, featuring starters and terminators, provides a powerful way to construct and deconstruct complex orders from simpler components, leading to the development of the sparse decomposition of interval orders Fahrenberg and Ziemiański (2024).

The primary objective of our research is to leverage this recent progress to develop an algorithm that systematically generates all non-isomorphic interval orders for a given number of elements. To achieve this, we introduce a new canonical notation based directly on the sparse decomposition. This canonical notation facilitates an efficient generation process that completely avoids the standard isomorphism problems encountered during the generation of standard POSETs.

This report first establishes the necessary background by defining POSETs and interval orders, including the key concepts of iposets and their decomposition. Then, it details the mathematical definitions of our new notation, presents theorems and proofs regarding its properties, and concludes by demonstrating how these theoretical results are applied to construct the generation algorithm.

Chapter 2

Preliminaries

This section recalls the fundamental definitions and concepts related to partially ordered sets (posets). These notions are essential for understanding the structure of interval orders and for formalizing the later constructions.

2.1 Posets

A partially ordered set (or poset) is a set that includes a partial order relation. For this reason, our first step is to define what a partial order is.

Definition 2.1.1 (Partial Order)**.** Let E be a set. A *partial order* on E is a binary relation $\leq \subseteq E \times E$ that is:

- *reflexive*: $\forall x \in E, x \leq x$,
- *antisymmetric*: $\forall (x, y) \in E^2, (x \leq y \wedge y \leq x) \Rightarrow x = y$,
- *transitive*: $\forall (x, y, z) \in E^3, ((x \leq y \wedge y \leq z) \Rightarrow x \leq z)$.

Definition 2.1.2 (Posets)**.** A Partially Ordered SET, or poset, is a pair $(E, \leq)$ where E is a set and $\leq$ is a partial order on E.

Definition 2.1.3. Hasse diagram representation Let $P = (E, \leq)$ be a poset. The Hasse diagram of P then follows these rules.

- Each even is represented by a point.
- For every pair of point (x, y) in P, $x < y \iff$ the point representing x is to the left of y.
- For every pair of point (x, y) in P, $x \to y \iff x < y \wedge (\nexists z \in E, x < z \wedge z < y)$

Example 2.1.1. Let's define the poset $P = (\{1, 2, 3, 5, 30\}, \leq)$ with $x \leq y \iff x$ divides y. This poset can be visually represented using its Hasse diagram as follows :

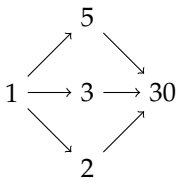

Figure 2.1: Example of a poset

This definition generalizes the idea of ordering elements without requiring every pair to be comparable. Indeed, we have order relations between 1 and 3 because 1 divides 3, but in this poset, 3 and 5 are not comparable.

Remark 2.1.1. When there is no risk of confusion, it is common practice to refer to the elements of a poset by the name of the poset itself.

Remark 2.1.2. From now on, for simplicity purposes, we will write $\neg(x \leq y)$ as $x \nleq y$.

Definition 2.1.4 (Comparability)**.** Let $P = (E, \leq)$ be a poset. Let $x, y \in E$. x and y are said to be comparable if $x \leq y$ or $y \leq x$. On the other hand, x and y are said to be incomparable if $x \nleq y$ and $y \nleq x$. We then write that $x \parallel y$

We will now introduce two important types of subsets within posets.

Definition 2.1.5 (Chain)**.** Let P be a poset. $C \subseteq P$ is a *chain* if every pair of elements in C is comparable. That is, $\forall x, y \in C$, $(x \leq y) \vee (y \leq x)$.

Definition 2.1.6 (Antichain)**.** Let P be a poset. $A \subseteq P$ is an *antichain* if every pair of distinct elements in A is incomparable. That is, $\forall x, y \in A,\ x \neq y \Rightarrow x \parallel y$

Chains represent totally ordered subsets, while antichains represent subsets with no order relation between their elements.

Definition 2.1.7 (Isomorphic Posets)**.** Two posets $(P, \leq_P)$ and $(Q, \leq_Q)$ are said to be *isomorphic* if there exists a bijection $f : P \to Q$ such that $\forall x, y \in P$, $x \leq_P y \Leftrightarrow f(x) \leq_Q f(y)$.

In most applications of posets, the specific labels of the elements are irrelevant. This is why isomorphic posets are structurally identical and thus often considered equivalent for classification purposes.

2.2 Interval orders

Definition 2.2.1 (Interval comparison)**.** Let $a, b, c, d \in \mathbb{R}$, $a < b$, $c < d$. Let I and J be intervals on $\mathbb{R}$ such that: $I = [a, b]$ and $J = [c, d]$

We write:

- $I = J \iff (a = c) \wedge (b = d)$
- $I < J \iff b < c$
- $I > J \iff a > d$

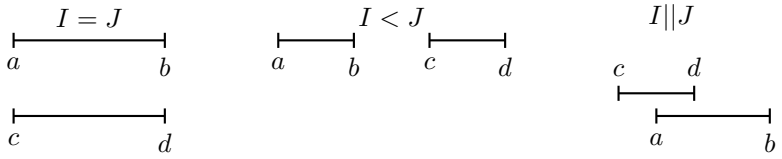

Figure 2.2: Various comparisons between intervals

- else, $I \parallel J$

Example 2.2.1. Here are some visual representations of comparisons between intervals

Definition 2.2.2 (Interval order)**.** Let $\mathcal{I}$ be the set of all intervals on $\mathbb{R}$. A poset $(E, \leq)$ is an interval order if and only if

$$\exists f : \begin{cases} P \to \mathcal{I} \\ e \mapsto I_e \end{cases}, \forall (x, y) \in P^2, x < y \Leftrightarrow I_x < I_y$$

Remark 2.2.1. To put it simply, a poset is an interval order if each element can be associated with an interval on $\mathbb{R}$ such that the ordering of the element in the poset is conserved in their interval representation.

Definition 2.2.3 (Fishburn Theorem)**.** A poset is an interval order if and only if it does not contain the 2+2 poset.

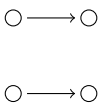

Figure 2.3: 2+2 poset

Remark 2.2.2. Visually, we can see that no matter how we try to arrange four intervals on $\mathbb{R}$, we can never get the ordering of a 2+2 poset. For example in the following figure, we can see that there is no way to fit the interval corresponding to the label d while respecting both $a \parallel d$ and $c \leq d$

a $\longrightarrow$ b

c $\longrightarrow$ d

Remark 2.2.3. Interval orders are therefore also generally referred to as 2+2 free posets.

Definition 2.2.4 (Interval orders set)**.** We will write the set of all Interval orders : O

Definition 2.2.5 (Minimal and maximal elements)**.** Let $P = (E, \leq)$ be a poset. We denote P^m the set of all minimal elements. An element x is said to be minimal if and only if

$$\forall y \in E, y \leq x \Rightarrow y = x$$

Similarly, we denote by P^M the set of all maximal elements, defined by

$$\forall y \in E, x \leq y \Rightarrow y = x$$

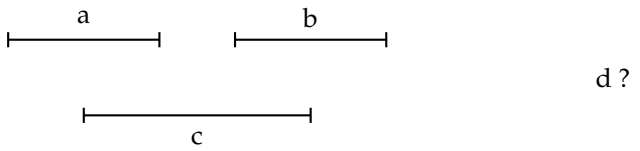

Figure 2.4: visual representation of the Fishburn Theorem

2.3 Posets with Interfaces

Posets with interfaces, or *iposets*, were introduced in this paper Fahrenberg et al. (2020)

2.3.1 Interfaces

We will now define another type of posets, called interfaces posets, or iposets for short.

Definition 2.3.1.1 (iposets)**.** An iposet is a structure $(E, \leq, S, T)$ where :

- E is a set of events,
- $\leq$ is a partial order on E,
- S is a subset $S \subseteq E^m$ called the source set,
- T is a subset $T \subseteq E^M$ called the target set.

Remark 2.3.1.1. Elements in the source set S can be interpreted as events that have already started at the beginning of the iposet, while elements in the target set T represent events that are still running at the end of the iposet.

To represent the source and target sets, we will use a point to the left or right of the event respectively.

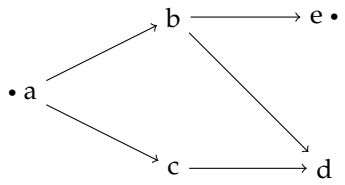

Figure 2.5: Example of a iposet where a is a source and e a target

2.3.2 Decomposition

Source and target elements of iposets are very useful tools when trying to compose iposets. Indeed, we can intuitively say that if a particular element is continuing at the end of an iposet and starting in another one, we can compose these two iposet to form a new one. Similarly, we can decompose iposets in two smaller ones by "splitting" an event between these two iposets.

Definition 2.3.2.1 (Discrete iposets)**.** An iposet $P = (E, \leq)$ is discrete if $\leq$ is empty.

Remark 2.3.2.1. Discrete iposets are also called "conc lists" because they can represent some events running concurrently with each other. They are the basic bricks that will be assembled to create larger posets. Also, since there are no relations between the elements of a discrete iposet, they can be written as a list of elements with their interfaces.

Example 2.3.2.1. The discrete iposet $(\{a, b, c\}, \emptyset, \{a, b\}, \{b, c\})$ will be written $\begin{pmatrix} \bullet a \\ \bullet b \bullet \\ c \bullet \end{pmatrix}$.

Definition 2.3.2.2 (Starter and Terminator)**.** An iposet $P = (E, \leq, S, T)$ is a Starter if it is discrete and $E = T$. Similarly, an iposet $P = (E, \leq, S, T)$ is a Terminator if it is discrete and $E = S$.

Definition 2.3.2.3 (Gluing composition)**.** Let $P_1 = (E_1, \leq_1, S_1, T_1)$ and $P_2 = (E_2, \leq_2, S_2, T_2)$ two iposets with $T_1 \cong S_2$. The iposet $P = (E, \leq, S, T)$ resulting of the gluing composition of P_1 and P_2 is defined by :

- $S = S_1$
- $T = T_2$
- The set of events E is constructed by taking the disjoint union of E_1 and E_2, and then identifying the corresponding elements of T_1 and S_2
- The order $\leq$ is defined as follows :
  Let $a \in P_i$ with $i \in \{1, 2\}$
  Let $b \in P_j$ with $j \in \{1, 2\}$
  $a \leq b \iff (i = j \wedge a \leq_i b) \vee (i < j \wedge a \notin T_1 \wedge b \notin S_2)$

Remark 2.3.2.2. In a gluing composition of iposet, the target elements of the first component must align with the source elements of the second. Intuitively, this means that events which are not completed at the end of the first iposet are assumed to continue into the beginning of the second. Additionally, any event that truly terminates in the first iposet (i.e., is not a target) is considered to precede any event that truly starts in the second (i.e., is not a source)

Definition 2.3.2.4 (Decomposition)**.** We say that P decomposes into iposets P_1 and P_2 if $P = P_1 \cdot P_2$.

Remark 2.3.2.3. Generally, we will decompose iposets into discrete iposets.

Example 2.3.2.2. Let $P = (\{a, b, c, d\}, \{(a, c), (b, d), (b, c)\}, \{a\}, \{d\})$. One of the decomposition of P into discrete iposets is :

$$(\bullet a \bullet) \cdot \begin{pmatrix} \bullet a \bullet \\ b \end{pmatrix} \cdot \begin{pmatrix} \bullet a \bullet \\ d \bullet \end{pmatrix} \cdot \begin{pmatrix} \bullet a \\ \bullet d \bullet \end{pmatrix} \cdot \begin{pmatrix} c \bullet \\ \bullet d \bullet \end{pmatrix} \cdot \begin{pmatrix} \bullet c \\ \bullet d \bullet \end{pmatrix}$$

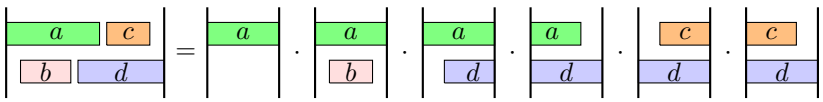

Figure 2.6: Visual representation of a decomposition into discrete iposets

Since there are infinitely many decompositions into discrete iposets for any non empty iposet, it will be useful to define a unique decomposition, called the sparse decomposition .

Definition 2.3.2.5 (Sparse decomposition)**.** A discrete iposet decomposition is said to be sparse if it alternates between starters and terminators. Each iposet has a unique sparse decomposition Fahrenberg and Ziemiański (2024).

Example 2.3.2.3. Let $P = (\{a, b, c, d\}, \{(a, c), (b, d), (b, c)\}, \{a\}, \{d\})$. The unique sparse decomposition of P is :

$$\binom{\bullet a \bullet}{b \bullet} \cdot \binom{\bullet a \bullet}{\bullet b} \cdot \binom{\bullet a \bullet}{d \bullet} \cdot \binom{\bullet a}{\bullet d \bullet} \cdot \binom{c \bullet}{\bullet d \bullet} \cdot \binom{\bullet c}{\bullet d \bullet}$$

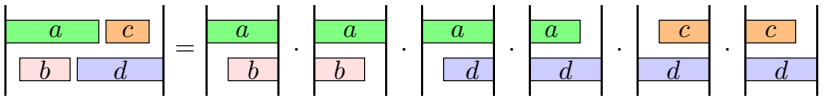

Figure 2.7: Visual representation of a sparse decomposition

Definition 2.3.2.6 (Iposet height)**.** We define the height of an iposet $P = (P, \leq_P, S_P, T_P)$ as the number of iposets that compose its sparse decomposition.
For example, the iposet $P = (\{a, b, c, d\}, \{(a, c), (b, d), (b, c)\}, \{a\}, \{d\})$ has a height of $h(P) = 6$, as seen in the example 1.3.3.

Definition 2.3.2.7 (Isomorphic Iposets)**.** Two iposets $(P, \leq_P, S_P, T_P)$ and $(Q, \leq_Q, S_Q, T_Q)$ are said to be *isomorphic* if there exists a bijection $f : P \to Q$ such that :

- $\forall x, y \in P, x \leq_P y \Leftrightarrow f(x) \leq_Q f(y)$
- $f(S_P) = S_Q$
- $f(T_P) = T_Q$

We then write $\mathcal{P} \cong \mathcal{Q}$

Chapter 3

Contributions

3.1 Isomorphism based on decomposition

Definition 3.1.1. Let an iposet $\mathcal{P} = (P, \leq_{\mathcal{P}}, S_{\mathcal{P}}, T_{\mathcal{P}})$. Let its sparse decomposition be $\mathcal{P} = \mathcal{P}_1 \cdot ... \cdot \mathcal{P}_n$. We will write :

- $N_{\mathcal{P}_i}$ the events started by $\mathcal{P}_i$, $\forall i \in [0, n]$
- $E_{\mathcal{P}_i}$ the events terminated by $\mathcal{P}_i$, $\forall i \in [0, n]$
- $\mathcal{P}^m$ the events $\leq_{\mathcal{P}}$ $-$minimal
- $\mathcal{P}^M$ the events $\leq_{\mathcal{P}}$ $-$maximal

Remark 3.1.1. In this context , $\forall i \in [0, n]$:

- $N_{\mathcal{P}_i} = T_{\mathcal{P}_i} \setminus S_{\mathcal{P}_i}$
- $E_{\mathcal{P}_i} = S_{\mathcal{P}_i} \setminus T_{\mathcal{P}_i}$
- $T_{\mathcal{P}_i} = (N_{\mathcal{P}_i} \cup S_{\mathcal{P}_i}) \setminus E_{\mathcal{P}_i}$
- $N_{\mathcal{P}_i} \cap S_{\mathcal{P}_i} = \emptyset$
- $E_{\mathcal{P}_i} \subseteq S_{\mathcal{P}_i}$
- If $\mathcal{P}_i$ is a starter then $T_{\mathcal{P}_i} = N_{\mathcal{P}_i} \cup S_{\mathcal{P}_i}$
- If $\mathcal{P}_i$ is a terminator then $T_{\mathcal{P}_i} = S_{\mathcal{P}_i} \setminus E_{\mathcal{P}_i}$ and $S_{\mathcal{P}_i} = T_{\mathcal{P}_i} \cup E_{\mathcal{P}_i}$

Remark 3.1.2. For all the following lemmas, theorems, and corollaries, we will use the following iposets: $\mathcal{P} = (P, \leq_{\mathcal{P}}, S_{\mathcal{P}}, T_{\mathcal{P}})$ and $\mathcal{Q} = (Q, \leq_{\mathcal{Q}}, S_{\mathcal{Q}}, T_{\mathcal{Q}})$

We will suppose that $\mathcal{P}$ and $\mathcal{Q}$ are isomorphic iposets, which means that there exists a bijection $f : P \to Q$ such that:

- $x \leq_{\mathcal{P}} y \Leftrightarrow f(x) \leq_{\mathcal{Q}} f(y)$
- $f(S_{\mathcal{P}}) = S_{\mathcal{Q}}$
- $f(T_{\mathcal{P}}) = T_{\mathcal{Q}}$

We will also define $\mathcal{P} = \mathcal{P}_1 \cdot ... \cdot \mathcal{P}_{n-1} \cdot \mathcal{P}_n$ and $\mathcal{Q} = \mathcal{Q}_1 \cdot ... \cdot \mathcal{Q}_{n-1} \cdot \mathcal{Q}_n$ their unique sparse decompositions.

Finally, we will write $\mathcal{P}' = \mathcal{P}_1 \cdot ... \cdot \mathcal{P}_{n-1}$ and $\mathcal{Q}' = \mathcal{Q}_1 \cdot ... \cdot \mathcal{Q}_{n-1}$

Theorem 3.1.1 (Lemma 1)**.** $\forall i \leq n, (\mathcal{P}_i \text{ starter } \Leftrightarrow \mathcal{Q}_i \text{ starter }) \wedge (\mathcal{P}_i \text{ terminator } \Leftrightarrow \mathcal{Q}_i \text{ terminator })$

Proof 3.1.1 (Lemma 1)**.** This proof demonstrates that isomorphism preserves the type (starter or terminator) of each component in a sparse decomposition. We will only prove the forward direction because the reverse follows by a symmetric argument.

Let us assume that $\mathcal{P}_i$ is a starter for some $i \in [1, n]$ The nature of the first component, $\mathcal{P}_1$, is determined by the parity of i.

- Case 1, i is even. In a sparse decomposition, components alternate in type. Since i is even, $\mathcal{P}_1$ and $\mathcal{P}_i$ must be of opposite types. Given that $\mathcal{P}_i$ is a starter, $\mathcal{P}_1$ must be a terminator. By definition, the source set of the first terminator in a decomposition consists of all minimal elements of the entire iposet, so $S_{\mathcal{P}_1} = \mathcal{P}^m$.
  The isomorphism f preserves both the minimal elements (so $f(\mathcal{P}^m) = \mathcal{Q}^m$) and the source sets of corresponding components. Therefore, $S_{\mathcal{Q}_1} = f(S_{\mathcal{P}_1}) = \mathcal{Q}^m$. An iposet whose source set contains all minimal elements is by definition a terminator, so $\mathcal{Q}_1$ is a terminator. Since the decomposition of $\mathcal{Q}$ also alternates and i is even, $\mathcal{Q}_i$ must be of the opposite type to $\mathcal{Q}_1$, meaning $\mathcal{Q}_i$ is a starter.

- Case 2, i is odd. If i is odd, $\mathcal{P}_i$ and $\mathcal{P}_1$ are of the same type. Thus $\mathcal{P}_1$ is also a starter. This implies that not all minimal elements of $\mathcal{P}$ are present in the source set S_{P_1}. Specifically, there exists at least one minimal element $m \in \mathcal{P}^m$ such that $m \notin S_{\mathcal{P}_1}$.
  The isomorphism f maps m to a minimal element $f(m) \in \mathcal{Q}^m$. Since $m \notin S_{\mathcal{P}_1}$ and f preserves source sets, we must have $f(m) \notin S_{\mathcal{Q}_1}$. The existence of a minimal element not in $S_{\mathcal{Q}_1}$ means that $\mathcal{Q}_1$ cannot be a terminator, and must therefore be a starter. As i is odd, $\mathcal{Q}_i$ is of the same type as $\mathcal{Q}_1$, which means $\mathcal{Q}_i$ is also a starter.

In both cases, if $\mathcal{P}_i$ is a starter, then $\mathcal{Q}_i$ is a starter. The proof for the terminator case follows the exact same logic.

Theorem 3.1.2 (Lemma 2)**.** Let $\mathcal{P}$ and $\mathcal{Q}$ be two isomorphic iposets. If their final components, $\mathcal{P}_n$ and $\mathcal{Q}_n$, are starters, then the isomorphism f maps the set of events of $\mathcal{P}_n$ to the set of events of $\mathcal{Q}_n$. That is, $f(P_n) = Q_n$.

Proof 3.1.2 (Lemma 2)**.** The proof relies on the property that for a starter iposet located at the end of a sparse decomposition, its set of events is identical to the total target set of the entire iposet. Since $\mathcal{P}_n$ is a final starter, we have $P_n = T_{\mathcal{P}_n} = T_{\mathcal{P}}$. Similarly, for $\mathcal{Q}_n$, we have $Q_n = T_{\mathcal{Q}_n} = T_{\mathcal{Q}}$.

The main isomorphism $f : \mathcal{P} \to \mathcal{Q}$ preserves the total target sets, meaning $f(T_{\mathcal{P}}) = T_{\mathcal{Q}}$. By substituting the identities above, we obtain:

$$f(P_n) = f(T_{\mathcal{P}}) = T_{\mathcal{Q}} = Q_n$$

Theorem 3.1.3 (Lemma 3)**.** Let $\mathcal{P} = \mathcal{P}_1 \cdot ... \cdot \mathcal{P}_n$ and $\mathcal{Q} = \mathcal{Q}_1 \cdot ... \cdot \mathcal{Q}_n$ be two isomorphic iposets. If their final components, $\mathcal{P}_n$ and $\mathcal{Q}_n$, are starters, then for any new event $x \in N_{\mathcal{P}_n}$, its image $f(x)$ is greater than or equal to any event b in the prefix $\mathcal{Q}' = \mathcal{Q}_1 \cdot ... \cdot \mathcal{Q}_{n-1}$. That is:

$$\forall x \in N_{\mathcal{P}_n}, \forall b \in Q \setminus Q_n, b \leq_{\mathcal{Q}} f(x)$$

Proof 3.1.3 (Lemma 3)**.** Let x be a new event introduced in the final component $\mathcal{P}_n$, so $x \in N_{\mathcal{P}_n}$. By the definition of the gluing composition, x is greater than or equal to any event not in the set P_n. Thus, for any event $c \in P \setminus P_n$, the relation $c \leq_{\mathcal{P}} x$ holds.

Now, let $b \in Q \setminus Q_n$. Let its preimage under the isomorphism be $c = f^{-1}(b)$.

Since $\mathcal{P}_n$ is a starter, we can use Lemma 2 to deduce that $f(P_n) = Q_n$. Since b is not in Q_n and f is a bijection, its preimage c cannot be in P_n. Therefore, $c \in P \setminus P_n$.

Since $c \in P \setminus P_n$ and $x \in N_{\mathcal{P}_n}$, we have $c \leq_{\mathcal{P}} x$ must be true. Applying the order-preserving isomorphism f yields:

$$f(c) \leq_{\mathcal{Q}} f(x)$$

By substituting $f(c)$ with b, we arrive at the conclusion: $b \leq_{\mathcal{Q}} f(x)$.

Theorem 3.1.4 (Lemma 4)**.** Let $\mathcal{P}$ and $\mathcal{Q}$ be two isomorphic iposets whose final components, $\mathcal{P}_n$ and $\mathcal{Q}_n$, are starters. Then the isomorphism f maps the set of new events of $\mathcal{P}_n$ to the set of new events of $\mathcal{Q}_n$. That is, $f(N_{\mathcal{P}_n}) = N_{\mathcal{Q}_n}$.

Proof 3.1.4 (Lemma 4)**.** We argue by contradiction.

Assume that the statement is false. This means there exists a new event $x \in N_{\mathcal{P}_n}$ whose image $f(x)$ is *not* a new event in $\mathcal{Q}_n$. By Lemma 2, since $x \in P_n$, we know $f(x) \in Q_n$. If $f(x)$ is not new within $\mathcal{Q}_n$, it must be an event that was already present, which means it belongs to the source set of $\mathcal{Q}_n$: $f(x) \in S_{\mathcal{Q}_n}$.

The source set of $\mathcal{Q}_n$ is the target set of the preceding component, $S_{\mathcal{Q}_n} = T_{\mathcal{Q}_{n-1}}$. Thus, $f(x)$ is an event present in the prefix $\mathcal{Q}' = \mathcal{Q}_1 \cdot \ldots \cdot \mathcal{Q}_{n-1}$. The component $\mathcal{Q}_{n-1}$ must be a terminator, as it precedes the starter $\mathcal{Q}_n$.

We have established two conflicting properties for the event $f(x)$:

1. Since $x \in N_{\mathcal{P}_n}$, Lemma 3 tells us that its image $f(x)$ must be greater than or equal to all events not in $\mathcal{Q}_n$.

2. Since $f(x)$ is an event within the terminator component $\mathcal{Q}_{n-1}$, and $f(x)$ itself isn't terminated by $\mathcal{Q}_{n-1}$, there must exist another element $z \in Q_{n-1}$ such that Q_{n-1} terminates z. Since z is terminated by Q_{n-1}, z is not in Q_n. But since z and $f(x)$ are both in Q_{n-1}, they are not comparable. Which contradicts the fact that $f(x)$ should be greater or equal than any element in $Q \setminus Q_{n-1}$.

Therefore, the assumption must be false, and we conclude that $f(N_{\mathcal{P}_n}) = N_{\mathcal{Q}_n}$.

Theorem 3.1.5 (Uniform isomorphism theorem)**.** If two iposets $\mathcal{P}$ and $\mathcal{Q}$ are isomorphic, then their prefixes $\mathcal{P}' = \mathcal{P}_1 \cdot \ldots \cdot \mathcal{P}_{n-1}$ and $\mathcal{Q}' = \mathcal{Q}_1 \cdot \ldots \cdot \mathcal{Q}_{n-1}$ are also isomorphic.

Proof 3.1.5 (Uniform isomorphism theorem)**.** We are given that $\mathcal{P} \cong \mathcal{Q}$ under a bijection f. We must show that $\mathcal{P}' \cong \mathcal{Q}'$ under f. Order preservation and source interface preservation ($f(S_{\mathcal{P}'}) = S_{\mathcal{Q}'}$) are direct consequences of the main isomorphism. We focus on proving the preservation of the event set and the target interface.

We proceed by analyzing the type of the final component, $\mathcal{P}_n$.

Case 1: $\mathcal{P}_n$ **is a starter.** By Lemma 1, $\mathcal{Q}_n$ is also a starter.

- **Event Set Preservation:** The set of events of the prefix, P', consists of all events in $\mathcal{P}$ except those newly introduced in $\mathcal{P}_n$. This can be written as the set difference $P' = P \setminus N_{\mathcal{P}_n}$. We can apply the bijection f to this relation:

$$f(P') = f(P \setminus N_{\mathcal{P}_n}) = f(P) \setminus f(N_{\mathcal{P}_n})$$

By hypothesis, $f(P) = Q$. And by Lemma 4, we know that for final starters, $f(N_{\mathcal{P}_n}) = N_{\mathcal{Q}_n}$. Substituting these gives:

$$f(P') = Q \setminus N_{\mathcal{Q}_n}$$

The set of events in $\mathcal{Q}$ minus the new events from its final component is precisely the definition of the event set of its prefix, Q'. Thus, $f(P') = Q'$.

- **Target Set Preservation:** The target set of the prefix, $T_{\mathcal{P}'}$, is identical to the source set of the next component, $S_{\mathcal{P}_n}$. From Lemma 2 and Lemma 4, we know that $f(P_n) = Q_n$ and $f(N_{\mathcal{P}_n}) = N_{\mathcal{Q}_n}$. The source set $S_{\mathcal{P}_n}$ is the set of non-new events in P_n, so $S_{\mathcal{P}_n} = P_n \setminus N_{\mathcal{P}_n}$. It follows that $f(S_{\mathcal{P}_n}) = f(P_n \setminus N_{\mathcal{P}_n}) = f(P_n) \setminus f(N_{\mathcal{P}_n}) = Q_n \setminus N_{\mathcal{Q}_n} = S_{\mathcal{Q}_n}$. Since $T_{\mathcal{Q}'} = S_{\mathcal{Q}_n}$, we have shown that $f(T_{\mathcal{P}'}) = T_{\mathcal{Q}'}$.

With all conditions met, we conclude that $\mathcal{P}' \cong \mathcal{Q}'$.

Case 2: $\mathcal{P}_n$ is a terminator. By Lemma 1, $\mathcal{Q}_n$ is also a terminator.

- **Event Set Preservation:** A terminator introduces no new events, so $N_{\mathcal{P}_n} = \emptyset$. This means the set of events in $\mathcal{P}$ is identical to that of its prefix ($P = P'$). Similarly, $N_{\mathcal{Q}_n} = \emptyset$ implies $Q = Q'$. Since we are given that $f(P) = Q$, it follows directly that $f(P') = Q'$.
- **Target Set Preservation:** First, we show that $f(S_{\mathcal{P}_n}) = S_{\mathcal{Q}_n}$. For a terminator, its source set is equal to its event set: $S_{\mathcal{P}_n} = P_n$. Furthermore, when a terminator is the final component, its events are the maximal elements of the entire iposet: $P_n = \mathcal{P}^M$. The isomorphism f preserves maximality, so $f(\mathcal{P}^M) = \mathcal{Q}^M$. This gives us the chain of identities:

  $$f(S_{\mathcal{P}_n}) = f(P_n) = f(\mathcal{P}^M) = \mathcal{Q}^M = Q_n = S_{\mathcal{Q}_n}$$

  So we have proven that the source sets of the final components are preserved.

  Now, we use the fundamental definition of decomposition. The target set of a prefix is the source set of the component that immediately follows it. Therefore:

  $$T_{\mathcal{P}'} = S_{\mathcal{P}_n} \quad \text{and} \quad T_{\mathcal{Q}'} = S_{\mathcal{Q}_n}$$

  Since we have just shown that $f(S_{\mathcal{P}_n}) = S_{\mathcal{Q}_n}$, we can substitute to get our result:

  $$f(T_{\mathcal{P}'}) = f(S_{\mathcal{P}_n}) = S_{\mathcal{Q}_n} = T_{\mathcal{Q}'}$$

  The target sets are therefore preserved.

Having proven the preservation of the event set and all interfaces, we conclude that $\mathcal{P}'$ and $\mathcal{Q}'$ are isomorphic.

Theorem 3.1.6 (Corollary). $\forall i \in [1, n]$, $\mathcal{P}_i$ and $\mathcal{Q}_i$ are isomorphic.

3.2 Event profiles

So far, we have used the sparse interval order decomposition. In the following, we are going to introduce what we call the profile of an event, which is determined by its position in the sparse decomposition. These profiles will be useful because of their canonical natures, indeed they do not rely on labels like the sparse decomposition could, and are therefore a great tool to manage posets isomorphism.

Definition 3.2.1 (Profile representation)**.** In an interval order, each event e can be represented by :

- $i_S(e)$ the index of its starter in the sparse decomposition.
- $i_T(e)$ the index of its terminator.

The event e can therefore be represented by the pair $\delta_e = (i_S(e), i_T(e))$. This pair is called the profile of e.

Definition 3.2.2 (Profile size)**.** The size of the profile of an event δ_e can be calculated as follows:

$$t(e) = \frac{i_T(e) - i_S(e) + 1}{2}$$

Definition 3.2.3 (Profile comparison)**.** We can now define the following order on these profiles. Let $P = (E, \leq)$ be a poset. Let $e_1, e_2 \in E$.

$$\delta_{e_1} <_\delta \delta_{e_2} \Leftrightarrow (t(e_1) < t(e_2)) \vee ((t(e_1) = t(e_2)) \wedge (i_S(e_1) < i_S(e_2)))$$

$$\delta_{e_1} =_\delta \delta_{e_2} \Leftrightarrow (i_S(e_1) = i_S(e_2)) \wedge (i_T(e_1) = i_T(e_2))$$

The resulting order $\leq_\delta$ is a total order on the set of all profiles.

Remark 3.2.1. More intuitively, to compare two profiles, we first compare them by their sizes and then by their starter index if their sizes are equal.

3.3 Interval order signature

The event profile gives us a canonical way to describe a single event. The logical next step is to extend this concept to represent an entire interval order. By collecting the profiles of all its events into a single object, we can create a unique identifier for the entire structure. This multiset of profiles, which we call the "signature", will not differ between two isomorphic interval orders. Also, two different signatures implies that the original interval orders are not isomorphic, which will be of a great help when generating them.

Definition 3.3.1 (Signature of an interval order)**.** Let $P = (E, \leq, S, T)$ an interval order. The signature of P is defined as the following multiset:

$$\Delta(P) = \{\delta_e \mid e \in E\}$$

Example 3.3.1. Let $P = (\{a, b, c, d, e\}, \{(a, c), (b, d), (b, c), (e, d)\}, \emptyset, \emptyset)$. The unique sparse decomposition of P is :

$$\begin{pmatrix} a\bullet \\ b\bullet \\ e\bullet \end{pmatrix} \cdot \begin{pmatrix} \bullet a\bullet \\ \bullet b \\ \bullet e \end{pmatrix} \cdot \begin{pmatrix} \bullet a\bullet \\ d\bullet \end{pmatrix} \cdot \begin{pmatrix} \bullet a \\ \bullet d\bullet \end{pmatrix} \cdot \begin{pmatrix} c\bullet \\ \bullet d\bullet \end{pmatrix} \cdot \begin{pmatrix} \bullet c \\ \bullet d \end{pmatrix}.$$

So we have:

- $\delta_a = (1, 4)$ and $t(\delta_a) = 2$
- $\delta_b = (1, 2)$ and $t(\delta_b) = 1$
- $\delta_c = (5, 6)$ and $t(\delta_c) = 1$
- $\delta_d = (3, 6)$ and $t(\delta_d) = 2$
- $\delta_e = (1, 2)$ and $t(\delta_e) = 1$

The signature of P is the multiset $\Delta(P) = \{(1, 2), (1, 2), (1, 4), (3, 6), (5, 6)\}$

Theorem 3.3.1 (Characterization of valid interval order signatures)**.** Let Δ be a multiset of profiles. Let $m = \max\{\, b \mid (a, b) \in \Delta \,\}$ be the maximum value in the signature.

Then Δ is the signature of an interval order if and only if every integer k with $1 \leq k \leq m$ appears in at least one profile, that is:

$$\forall k \in \{1, 2, \ldots, m\}, \quad \exists\, (a, b) \in \Delta \text{ such that } k = a \text{ or } k = b.$$

Proof 3.3.1. We will prove the forward direction by contradiction. Let $\mathcal{P} = (P, \leq)$ be an interval order with a valid signature $\Delta(\mathcal{P})$. Let $m = h(\mathcal{P})$ be its height, which corresponds to the maximum index appearing in $\Delta(\mathcal{P})$. The sparse decomposition of $\mathcal{P}$ is $\mathcal{P} = \mathcal{P}_1 \cdot \mathcal{P}_2 \cdots \mathcal{P}_m$.

Assume, for the sake of contradiction, that there exists an integer $k \in \{1, 2, \ldots, m\}$ that does not appear in any profile in $\Delta(\mathcal{P})$. This means that for every event $e \in P$, its profile $(i_S(e), i_T(e))$ is such that neither $i_S(e)$ nor $i_T(e)$ is equal to k.

This has a direct consequence on the k-th component of the sparse decomposition, $\mathcal{P}_k$. The set of events starting in $\mathcal{P}_k$ is $N_{\mathcal{P}_k}$, and the set of events terminating in $\mathcal{P}_k$ is $E_{\mathcal{P}_k}$. Our assumption implies that no event starts or terminates at index k, so both $N_{\mathcal{P}_k}$ and $E_{\mathcal{P}_k}$ must be empty.

However, every component in a sparse decomposition must be either a starter (requiring $N_{\mathcal{P}_k} \neq \emptyset$) or a terminator (requiring $E_{\mathcal{P}_k} \neq \emptyset$). A component where both sets are empty has no defined role and violates this fundamental property of the sparse decomposition.

This contradiction proves that our initial assumption was false. Therefore, every integer from 1 to m must appear at least once in the signature, either as a starting or a terminating index.

Let's now prove the other direction. Let Δ be a multiset of profile with the appearnce of every integer k with $1 \leq k \leq m$ in at least one profile. Let $P_1 \cdot P_2 \cdot ... \cdot P_{k-1} \cdot P_k$ be a sparse decomposition corresponding to Δ.
Then for every odd number $i \in [1, k-1]$, there exists a profile $(a, b) \in \Delta$ such that $a = i$, and there exists no profile $(c, d) \in \Delta$ such that $d = i$, because by the profile definition, d is even.
Therefore, P_i starts at least one event and terminates none. So for every odd number i, P_i is a starter
Similarly for every even number $j \in [2, k]$, there exists a profile $(a, b) \in \Delta$ such that $b = j$, and there exists no profile $(c, d) \in \Delta$ such that $c = j$, because by the profile definition, c is odd.
Therefore, P_j terminates at least one event and starts none. So for every odd number j, P_j is a terminator
Since the decomposition alternates between starters and terminators, it is a valid sparse decomposition and there exists an interval order with such decomposition.

Remark 3.3.1. Since the maximum value occurring in the signature of an interval order $P = (E, \leq)$ is the index of the last terminator in the sparse decomposition, which himself is equal to $h(P)$, we can therefore deduce that every number between 1 and $h(P)$ must appear in the signature.

Theorem 3.3.2 (Corollary)**.** Let $P = (E, \leq)$ be an interval order with signature $\Delta(P) = \{\delta_e \mid e \in E\}$. Then $Card(\Delta(P)) \geq h(P)/2$

3.4 Generation by prototype

This section presents a generation based on what we call prototypes, which are interval orders that can be easily generated and then used to generate the whole set of interval orders according to simple rules. Generating a list of prototypes for a given number of event is easier than generating every possible interval order, and we will use some particular rules to then derivate interval orders from these prototypes.

3.4.1 Representation of interval order in matrices

To make it easier to manipulate interval orders and their signatures, we will introduce a matrix notation. Each cell represents a particular possible profile and the number in each cell represents how many time the profile is in the signature.

Definition 3.4.1.1 (Interval Order Matrix)**.** Let $P = (E, \leq)$ be an interval order. Its matrix M_P is a matrix where:

- Rows correspond to profiles sizes, starting from 1.
- Columns correspond to starter positions, starting from 1 and then going through every odd number.
- The entry $M_P(s, p)$ represents events whose profile is $(2p - 1, 2(p + s - 1)$.

	$starter\,1$	$starter\,3$	$starter\,5$	$\cdots$
$size\,1$	$(1,2)$	$(3,4)$	$(5,6)$	$\cdots$
$size\,2$	$(1,4)$	$(3,6)$	$(5,8)$	$\cdots$
$size\,3$	$(1,6)$	$(3,8)$	$(5,10)$	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Figure 3.1: The different profiles in the matrix

Remark 3.4.1.1. Increasing the profile size reduces the number of events that can be placed before reaching the maximum terminator index. Consequently, the resulting matrices are necessarily upper-left triangular.

This representation allows interval orders to be manipulated directly as matrices of numbers, instead of a list of profiles, which allows to check the existence of a particular profile way more easily, alongside with other optimisations.

Example 3.4.1.1. Let P be an interval order and $\Delta(P) = \{(1,2),(3,4),(3,4),(5,8),(3,6),(7,8)\}$. The matrix representation of this interval order will be :

	$starter\,1$	$starter\,3$	$starter\,5$	$starter\,7$
$size\,1$	1	2	0	1
$size\,2$	0	1	1	
$size\,3$	0	0		
$size\,4$	0			

Remark 3.4.1.2. For an interval order P, every integer between 1 and $h(P)$ must appear in the signature $\Delta(P)$ — either as a starting or a terminating index. This implies that all odd numbers in this range (i.e., $1, 3, \ldots, h(P) - 1$) must appear as starters, and all even numbers (i.e., $2, 4, \ldots, h(P)$) must appear as terminators.

Since each column in the interval order matrix corresponds to a starter index, and each diagonal corresponds to a terminator, this means that no entire row or diagonal of the matrix can be filled with zeros. In other words, to represent a valid interval order, the matrix must have at least one nonzero entry in every row and every diagonal.

3.4.2 Prototype

Prototypes serve as the minimal building blocks of interval orders. These are irreducible structures whose removal of any maximal profile breaks their validity. Using such prototypes, we can generate all possible interval orders through derivations.

Definition 3.4.2.1 (Prototype)**.** Let $P = (E, \leq)$ be an interval order with signature $\Delta(P) = \{\delta_e \mid e \in E\}$.

We say that P is a *prototype* if the following two conditions hold:

1. **Irreducibility:** For all interval orders Q,

$$\Delta(Q) \neq \Delta(P) \setminus \{\max(\Delta(P))\}$$

   That is, removing the maximal profile from $\Delta(P)$ yields a multiset that is not the signature of any other interval order.

2. **Injectivity:** The signature of P is injective:

$$\forall e, f \in E,\ e \neq f \Rightarrow \delta_e \neq \delta_f$$

We will now first describe the algorithm for the generation of prototypes, and then the algorithm for the generation of interval orders from these prototypes.

3.4.3 Prototype generation

The prototype generation algorithm will be given 5 arguments, the first one being the maximum number of events allowed to create the prototypes, written s, the second one being a matrix of size $h/2 * h/2$ with h being the wanted height of the generated prototypes, filled with zeros. For a height h, there are h/2 possible odd-numbered starter positions (1, 3, ..., h-1), so the matrix is of size h/2 × h/2. The third one is a list containing the results (initialized to []), the fourth being the current i coordinate in the matrix (initialized to 0) and the fifth the current j coordinate in the matrix (initialized to 0).

Remark 3.4.3.1. This algorithm is unoptimized and is not the exact one we used to generate our prototypes, but this is the core algorithmic idea of the generation.

Proof 3.4.3.1 (Good functioning of prototype generation)**.** Let $P = (E, \leq_P)$ with a signature $\Delta(P) = \{\delta_e \mid e \in E\}$ be a prototype, with $len(M) = h(P)/2$, M being the interval order matrix passed to the function
Then $\Delta(P) = \{\delta_1, \delta_2, ..., \delta_n\}$ with $n = Card(E), \delta_i <_\delta \delta_{i+1}, \forall i \in \{1, ..., n\}$, because each profile

Algorithm 1 Prototype generation

```
function GeneratePrototypes(s,result,M,i,j)
    if s < 0 then
        return
    if sum of every diagonal and sum of every column of M ≠ 0 then
        result ← result ∪ [M]
        return
    if i + j ≥ len(M) then
        i = i + 1
        j = 0
    if i ≥ len(M) then
        return
    GeneratePrototypes(s,result, copy(M), i, j + 1 )
    M[i,j] = 1
    GeneratePrototypes(s - 1,result, copy(M), i, j + 1)
```

appears only one time in the decomposition due to the fact that P is a prototype, which allows them to be ordered.
The algorithm fills the matrix in ascending order of profiles, which means that $\forall i, j \in \{1, ..., n\}, \delta_i < \delta_j \Leftrightarrow \delta_i$ will be filled before δ_j

Let $\delta_e = (i_s(e), i_t(e)) \in \Delta(P)$, the corresponding i, j indexes in the matrix are $i = \frac{i_s(e)-1}{2}$ and $j = \frac{i_T(e)-i_S(e)+1}{2} - 1$ (because we are 0 indexed)

we have $i_t(e) \leq h(p)$, so : $i + j = \frac{i_T(e)}{2} - 1 \leq \frac{h(p)}{2} - 1 < \frac{h(p)}{2} < len(M)$

Which means that if we get to the indexes $i = \frac{i_s(e)-1}{2}$ and $j = \frac{i_T(e)-i_S(e)+1}{2} - 1$, we won't go in the 3rd and 4th if conditions and will fill recursively the matrix with a 1 at this place.

Also, since P is a prototype, for all interval orders Q, $\Delta(Q) \neq \Delta(P) \setminus \{\max(\Delta(P))\}$, so we won't get in the second if condition and add the result until $\max(\Delta(P))$ is filled in the matrix. This means that $\forall \delta \in \Delta(P) \setminus \{max(\Delta(P))\}$, we will reach the index corresponding to δ without returning the result and stopping the recursive call before hand.
But since the algorithm fills the matrix in ascending order, $max(\Delta(P))$ will be filled last.
At this point, since P is a prototype, the sum of every diagonal and every column will be different than 0, which allows us to return P.

Furthermore, each signature will be met only one time, because of the completeness of the recursive exploration and the fact that we only go up in indexes.

3.4.4 Interval order generation

Now that we have the possibility to generate every prototype for a given amount of points, we are able to generate every interval order from these prototypes. In order to do so, we first have to introduce the notion of "root", which is the prototype from a particular interval order is derived.

Definition 3.4.4.1 (Interval order root)**.** A prototype $P = (E, \leq_P)$ with a signature $\Delta(P) = \{\delta_e \mid e \in E\}$ is said to be the root of an interval order $Q = (F, \leq_Q)$ with a signature $\Delta(Q) = \{\delta_f \mid f \in F\}$ if and only if

- $\Delta(P) \subset \Delta(Q)$
- $\forall \delta_f \in \Delta(Q), \delta_f \notin \Delta(P) \Rightarrow max(\Delta(P)) \leq_\delta \delta_f$
- $h(P) = h(Q)$

Remark 3.4.4.1. Intuitively, a prototype P is the root of an interval order Q if every profile in the signature of P is in the signature of Q, and every new profile of Q must be greater than the biggest profile of P.

Theorem 3.4.1 (Existence and uniqueness of the root)**.** Let $Q = (F, \leq_Q)$ be an interval order with a signature $\Delta(Q) = \{\delta_f \mid f \in F\}$. Then $\exists!$ prototype $P = (E, \leq_P)$ such that P is the root of Q

Proof 3.4.4.1 (Existence)**.** Let $Q = (F, \leq_Q)$ be an interval order with a signature $\Delta(Q) = \{\delta_f \mid f \in F\}$.
We construct the set $\Delta = \{\delta_f \mid f \in F\}$, removing the duplicates.
Then, because the order on profiles is total, we can order each profile in Δ such that :
$\Delta = \{\delta_1, \delta_2, ..., \delta_n\}$ with $n = Card(\Delta), \delta_i <_\delta \delta_{i+1}, \forall i \in \{1, ..., n\}$
Let $\Delta_i = \{\delta_1, ..., \delta_i\}$, be the set of profiles labeled 1 to i
Let $E = \{i \in \{1, ..., n-1\} \mid \nexists P$ an interval order such that $\Delta(P) = \Delta_i$ and $h(P) = h(Q)\}$
E is then the set of indexes such that for any index $i \in E$, the resulting signature Δ_i is not valid.
E is not empty because $h(Q)/2 - 1 \in E$, since every interval order of height h must contain at least $h/2$ profiles. So $\Delta_{h(Q)/2-1}$ would not be valid.
Let $k = max(E) + 1$
Then k is not in E.
Therefore $\exists P$ an interval order such that $\Delta(P) = \Delta_k$ and $h(P) = h(Q)$
By construction, $\Delta(P) \subset \Delta(Q)$
The only property left to prove that $\Delta(P)$ is indeed a root of Q, is that every profile in $\Delta(Q)$ which is not in $\Delta(P)$ is greater than the maximum profile of $\Delta(P)$.
The maximum profile of $\Delta(P)$ is δ_k because any profile labeled with an index higher than k is not in Δ_k, which is $\Delta(P)$
Every profile of $\Delta(Q)$ not in $\Delta(P)$ is greater than δ_k, because otherwise such profile would be included in Δ_k.
Therefore, P is a root of Q

Proof 3.4.4.2 (Uniqueness)**.** Let $Q = (F, \leq_Q)$ be an interval order with a signature $\Delta(Q) = \{\delta_f \mid f \in F\}$.
Let $P = (E, \leq_P)$ be a root of Q with a signature $\Delta(P) = \{\delta_e \mid e \in E\}$.
Let $R = (G, \leq_R)$ be a root of Q with a signature $\Delta(R) = \{\delta_g \mid g \in G\}$.
We chose P and R such that the maximum profile of $\Delta(R)$ is greater or equal to the maximum profile of $\Delta(P)$
Since R and P are roots of Q, they have the same height, so we have $h(P) = h(R) = h(Q)$
Let's argue by contradiction and suppose that $P \neq R$
P and R are prototypes, so each profile is unique in the signature and we can order them.
Let $\Delta(P) = \{\delta_{P_1}, \delta_{P_2}, ..., \delta_{P_n}\}$ with $n = Card(E)$ and $\delta_{P_i} <_\delta \delta_{P_{i+1}}, \forall i \in \{1, ..., n\}$
Let $\Delta(R) = \{\delta_{R_1}, \delta_{R_2}, ..., \delta_{R_m}\}$ with $m = Card(G)$ and $\delta_{R_i} <_\delta \delta_{R_{i+1}}, \forall i \in \{1, ..., m\}$
$P \neq R$ so there either exists a profile $\delta_{P_k} \in \Delta(R)$ such that $\delta_{P_k} \notin \Delta(R)$ or the other way around.

Let's first suppose that there exists a profile $\delta_{P_k} \in \Delta(P)$ such that $\delta_{P_k} \notin \Delta(R)$
Since $\delta_{P_k} \in \Delta(P)$, δ_{P_k} is less or equal to the maximum profile of $\Delta(P)$.

The maximum profile of $\Delta(P)$ is itself less or equal to the maximum profile of $\Delta(R)$, so δ_{P_k} is less or equal to the maximum profile of $\Delta(R)$
Since P is a root of Q, $\Delta(P) \subset \Delta(Q)$ so $\delta_{P_k} \in \Delta(Q)$
But R is also a root of Q, so every profile in $\Delta(Q)$ not in $\Delta(R)$ should be greater or equal to the maximum profile of $\Delta(R)$.
δ_{P_k} is in $\Delta(Q)$, is not in $\Delta(R)$ and is not greater or equal to maximum profile of $\Delta(R)$
Which means that R is not a root of Q, which is a contradiction.

Let's now suppose that there exists a profile $\delta_{P_k} \in \Delta(R)$ such that $\delta_{P_k} \notin \Delta(P)$
If δ_{P_k} is less or equal to the maximum profile of $\Delta(P)$, the same contradiction arises.
Let's study the case of when δ_{P_k} is greater or equal to the maximum profile of $\Delta(P)$.
Since P is a root of Q, every profile in $\Delta(P)$ must be in $\Delta(Q)$.
Since R is also a root of Q, every profile in $\Delta(P)$ and $\Delta(Q)$ must either be in $\Delta(R)$, or be greater than the maximum profile of $\Delta(R)$.
But every profile of $\Delta(P)$ is less or equal to the maximum profile of $\Delta(R)$, so every profile of $\Delta(P)$ is also in $\Delta(R)$.
Which means that $\Delta(P) \subset \Delta(R)$, and contradicts the fact that R is a prototype because of the irreducibility property.
So $\Delta(P)$ must be equal to $\Delta(R)$, and the root is unique.

Example 3.4.4.1. Let P be a prototype and $\Delta(P) = \{(1,2),(3,4),(7,8),(3,6),(5,8)\}$. The matrix representation of this prototype will be :

	$starter\,1$	$starter\,3$	$starter\,5$	$starter\,7$
$size\,1$	1	1	0	1
$size\,2$	0	1	1	
$size\,3$	0	0		
$size\,4$	0			

Let Q be the interval order and $\Delta(Q) = \{(1,2),(3,4),(3,4),(7,8),(3,6),(5,8)\}$. The
The matrix representation of this interval order will be :

	$starter\,1$	$starter\,3$	$starter\,5$	$starter\,7$
$size\,1$	1	2	0	1
$size\,2$	0	1	1	
$size\,3$	0	0		
$size\,4$	0			

Let R be the interval order and $\Delta(R) = \{(1,2),(3,4),(7,8),(3,6),(5,8),(1,8)\}$. The
The matrix representation of this interval order will be :

	$starter\,1$	$starter\,3$	$starter\,5$	$starter\,7$
$size\,1$	1	1	0	1
$size\,2$	0	1	1	
$size\,3$	0	0		
$size\,4$	1			

The root of Q and R is the prototype P

Remark 3.4.4.2. Visually, to go from an interval order to its root, we have to delete duplicates and remove the biggest profile of the signature while the resulting signature is valid. If removing the biggest profile from the remaining signature creates an invalid one, then the current signature is the signature of the root.

The generation of all interval orders from a given root prototype is performed by a recursive algorithm. It takes 5 arguments: the matrix of the root prototype root, the number of additional profiles s to add, the result list, and the current matrix coordinates i and j. The procedure is detailed in Algorithm 2. This algorithm will be used after getting all prototypes of a given size with Algorithm 1.

Algorithm 2 Root derivation

```
function FINDMAXINDEXES(root)
    resI ← 0
    resJ ← 0
    for i ← 0 to len(root) - 1 step 1 do
        for j ← 0 to len(root) - i - 1step 1 do
            if root[i,j] ≠ 0 then
                resI ← i
                resJ ← j
    return resI, resJ
function IOFROMROOT(root,s, result, i, j)
    maxI, maxJ ← FindMaxIndexes(root)
    if s ≤ 0 then
        result ← result ∪ copy(root)
        return
    if i + j ≥ len(root) then
        i = i + 1
        j = 0
    if i ≥ len(root) then
        return
    if root[i,j] = 1 or  i > maxI or (i = maxI and j ≥ maxJ) then
        root[i,j] ← root[i,j] + 1
        IOFromRoot(root,s-1,result,i,j)
        root[i,j] ← root[i,j] - 1
    IOFromRoot(root,s-1,result,i,j)
```

Remark 3.4.4.3. Once again, this algorithm is not optimized and the version we used to generate our results is a lot more efficient.

Proof 3.4.4.3 (Good functioning of interval order generation)**.** Let $P = (E, \leq_P)$ with a signature $\Delta(P) = \{\delta_e \mid e \in E\}$ be an interval order.
Let $R = (F, \leq_R)$ with a signature $\Delta(R) = \{\delta_f \mid f \in F\}$ be the root of P.

$s = Card(\Delta(P)) - Card(\Delta(R))$, representing the amount of profiles having yet to be generated.
In order to obtain P in the result, we pass s and R as parameters in the function.

While $s > 0$, the interval order continues to be filled, which means that we will only stop when we reach exactly the good number of profiles and we add the result at this moment.

Similarly than the last algorithm, since the size of the matrix matches with the height of P, the indexes of every profile in $\Delta(P)$ is valid and will be reached eventually

Since R is the root of P, $\forall \delta_e \in \Delta(P)$, $\delta_e \in \Delta(R)$ or $\delta_e \geq_\delta max(\Delta(R))$
If $\delta_e \in \Delta(R)$, then the corresponding place in the matrix is already filled with a 1 and so the condition in the if is respected.
If $\delta_e \geq_\delta max(\Delta(R))$, then we have the variables $maxI$ and $maxJ$ that are the indexes of

$max(\Delta(R))$. Since $\delta_e \geq_\delta max(\Delta(R))$, the indexes of δ_e are also bigger, so the condition is also met.

Furthermore, each signature will be met only one time, because of the completeness of the recursive exploration and the fact that we only go up in indexes.

Chapter 4

Conclusion

In this research, we first described the mathematical concepts of POSETs, interval orders and the sparse decomposition.

We then proved that an isomorphism on two interval orders implies that the interval orders composed of every component iposet in their sparse decomposition except the last are also isomorphic. This implies that every sub interval order in one of the sparse decomposition is isomorphic to the corresponding sub interval in the second decomposition, which creates an additional useful property on interval orders isomorphism.

Subsequently, we introduced our new notation of interval orders based on the sparse decomposition that doesn't use the labels of the original orders. In order to do so, we first introduced the concept of event profiles to reduce an event to a pair of indexes, and then the concept of interval order signature. This signature is composed of the profiles of all events belonging in the interval order. Finally, we introduced the concept of prototypes, which are a subset of interval orders that can be derived to obtain every interval order.

After that, we proved some useful properties of these new objects and introduced a way to efficiently store these objects in the form of triangular matrices. This allowed the conception of an algorithm using these properties and the matrices. This algorithm is separated into two parts: the generation of prototypes, and the creation of interval orders from these prototypes. We then proved that these algorithms actually work, using the properties of prototypes, which allow us to be sure that each interval order with a given amount of points will be generated by our algorithm, and exactly once. With these algorithms, we were able to generate every interval order composed of less than 14 events, in approximately 20 minutes, using an implementation in Python, which is a satisfactory result.

It is also worth mentioning that the matrix representation of interval orders had already been explored by Fishburn in his book : Fishburn (1985), but he never mentioned an algorithm to generate interval orders based on these matrices.

Also, through the OEIS we noticed that there are objects similar to interval orders, so-called ascent sequences; in fact Bousquet-Mélou et al. (2009) show that these two structures are isomorphic. The generation of ascent orders is faster than our methods, but the transformation of ascent orders into interval orders is still costly. That's why we are not sure about the efficiency gains between our generation method and a method that would involve generating ascent orders to then translate them into interval orders. Nonetheless, this would be possibly a way to improve our results even more and establish links between these two fields.

Chapter 5

Bibliography

Bousquet-Mélou, M., Claesson, A., Dukes, M., and Kitaev, S. (2009). (2+2)-free posets, ascent sequences and pattern avoiding permutations. (page 25)

Fahrenberg, U., Johansen, C., Struth, G., and Thapa, R. B. (2020). Generating Posets Beyond N. In Fahrenberg, U., Jipsen, P., and Winter, M., editors, *Relational and Algebraic Methods in Computer Science : 18th International Conference, RAMiCS 2020, Palaiseau, France, October 26–29, 2020, Proceedings*, volume 12062, pages 82–99, Palaiseau, France. Springer. (pages 4 and 8)

Fahrenberg, U. and Ziemiański, K. (2024). Myhill-nerode theorem for higher-dimensional automata. (pages 4 and 10)

Fishburn, P. (1985). *Interval Orders and Interval Graphs: A Study of Partially Ordered Sets*. A Wiley-Interscience publication. Wiley. (pages 4 and 25)